Thematic Assessment Reports
Phases 1 to 3

TA REPORTS · PHASES 1 TO 3

Persistent Memory Architecture in Agents Using DBMS

Proposal · Design · Implementation

Submitted in partial fulfilment of course requirements
for Database Management Systems

Bachelor of Technology
Computer Science and Engineering
School of Engineering and Technology

Submitted to:
Dr. Poonam Sangwan
Faculty, DBMS
Department of CSE / SOET
Date of submission: _____

Submitted by:
Harshit Khemani (241302081)
Kush Ahuja (241302122)
Kushagra Agrawal (241302109)
Madhav Bassi (241302100)
B.Tech CSE (AI/ML), Section C

https://github.com/HKTITAN/dbms-agent-memory

SGT University
School of Engineering and Technology

Contents

TA Report Phase 1: Abstract and Project Proposal **3**

Abstract 4

1. Introduction and problem statement 4

2. Motivation 4

3. Literature review 5

4. Objectives 5

5. Scope and intended users 5

6. Proposed tools 5

7. Preliminary entities 6

8. Plan of work 6

References 6

TA Report Phase 2: Survey, Requirements, and Database Design **8**

1. Introduction and objective 9

2. Memory operations 9

3. Related memory systems 9

4. One logical schema on three substrates 10

5. Requirements 10

6. Conceptual design 11

7. Logical design 13

8. Functional dependencies and normalisation 14

9. Physical design plan 16

10. Recommendation and plan for Phase 3 16

References 17

TA Report Phase 3: Implementation (Python · SQLite · FTS5) **18**

1. Introduction and objective 19

2. Software and tools 19

3. System architecture 19

4. Database schema 20

5. Implementation 22

6. Output and results 24

7. Testing and observations 29

8. Limitations and future work 30

9. Conclusion 30

References 30

Page numbers above are pages of this combined PDF (31 pages). Each report keeps its own cover and its own page numbers in the footer.

Thematic Assessment Report
Phase 1

TA REPORT PHASE 1

Persistent Memory Architecture in Agents Using DBMS

Abstract and Project Proposal

Submitted in partial fulfilment of course requirements
for Database Management Systems

Bachelor of Technology
Computer Science and Engineering
School of Engineering and Technology

Submitted to:
Dr. Poonam Sangwan
Faculty, DBMS
Department of CSE / SOET
Date of submission: _____

Submitted by:
Harshit Khemani (241302081)
Kush Ahuja (241302122)
Kushagra Agrawal (241302109)
Madhav Bassi (241302100)
B.Tech CSE (AI/ML), Section C

https://github.com/HKTITAN/dbms-agent-memory

SGT University
School of Engineering and Technology

Declaration. We declare that this proposal is our own work for DBMS TA-1.

Abstract

An AI agent that works with a team over many sessions has to remember what it learned: who owns a project, which database an application uses, who wrote a schema, and when any of that changed. Most agent memories keep this as free text in a file or as vectors in an index. Neither can declare a key, a foreign key, or a rule such as "a project has one current owner". This project treats agent memory as a database problem. We review how persistent memory for AI agents is built today, and we propose a relational memory store with two kinds of memory: episodic memory (what was said, in which session and turn) and semantic memory (facts stored as subject, predicate and object, with the time interval in which each fact held). The store has to support five operations on one schema with keys and checks: write, retrieve, update, forget, and consolidate. We plan to build it on SQLite with full-text search, so that it runs from a clone without a database server.

Keywords. agent memory · relational model · entity-relationship model · normalisation · integrity constraints · valid time · full-text search · SQLite

1. Introduction and problem statement

A language-model agent forgets everything when its context is cleared. Memory systems for agents store selected parts of past sessions outside the model and load them back when they are needed. Such a store is a database, but it is rarely designed as one. The same project can be called "Campus Navigator", "navigator" and "the app". A fact such as "the DBMS project team owns Campus Navigator" can stop being true. A claim can be a wrong guess that should be forgotten without losing the record of where it came from.

Problem. Design and implement a persistent memory for one AI agent in which these situations are handled by the database schema (keys, foreign keys, check constraints, and a validity interval) rather than by convention in application code.

2. Motivation

Each requirement of an agent memory has a standard database counterpart. Table 1 lists the ones this project depends on. Each row becomes a design decision in Phase 2.

Agent-memory need	DBMS concept
Keep what was learned after the session ends	Persistent storage in one database file
Recognise the same thing under different names	An entity table plus an alias table with a unique key
A fact changes over time (ownership moves)	Valid time: valid_from and valid_until on each fact
Know why the agent believes something	Provenance: a foreign key from a fact to the episode that supports it
Only one current owner of a project	A uniqueness constraint over current facts
Find memories by the words they contain	Full-text and keyword indexes
Forget a wrong memory but keep the audit trail	A status flag instead of a DELETE, protected by foreign keys
Summarise a session	A stored summary row: a materialised query result

Table 1: What an agent memory needs, and the DBMS concept that provides it.

3. Literature review

Du and colleagues describe memory for AI systems through six atomic operations: consolidation, indexing, updating, forgetting, retrieval, and compression [1]. MemGPT manages memory in tiers, like an operating system that pages data between the context window and external storage [2]. Mem0 extracts memories from a conversation and then decides whether to add, update, or delete stored entries [3]. Zep keeps memory as a temporal knowledge graph, so a relationship records when it was valid [4]. Lore, from Notion, keeps an agent's memory as pages in Notion databases, so that people can read and edit it [5].

These systems are mostly described in machine-learning terms: what to extract, how to rank, and when to summarise. The guarantees a DBMS gives are usually not stated: a key that prevents a duplicate alias, a foreign key that protects provenance, and a constraint that prevents two current owners. The relational model [6] and the entity-relationship model [7] give the vocabulary for stating them, and SQL:2011 standardised valid-time periods [8]. This project applies that vocabulary to agent memory. Phase 2 extends the survey and compares the substrates.

4. Objectives

1. Review persistent memory for AI agents and identify the memory operations a store must support.
2. Gather requirements and design a conceptual (ER) model with episodic and semantic memory.
3. Map the ER model to relational tables and normalise them, stating keys and functional dependencies.
4. Implement the schema in SQLite with primary keys, foreign keys, checks, and full-text search.
5. Implement write, retrieve (keyword and vector), update, forget, and consolidate in a small program.
6. Show one scripted agent session, with checks a reader can run again.

5. Scope and intended users

In scope	Out of scope
One agent, its sessions, and the turns it records (episodes)	Many concurrent writers and distributed storage
Entities with canonical names and aliases	A neural embedding model or a hosted vector database
Facts as subject, predicate, object with a validity interval	A language model that writes summaries
Keyword, full-text, and simple vector retrieval	Authentication and multi-user access control
Supersede, forget, consolidate, as-of queries	A graphical user interface

Table 2: Scope of the project.

Intended users. The direct user is an AI agent (a program that calls the store during a session). The people who use it are developers who build agents and need to inspect or correct what an agent remembers, and students of DBMS who want a small worked example of keys, constraints, normalisation, and temporal data.

6. Proposed tools

Software	Role
Python 3	Implementation language. `sqlite3` is in the standard library.
SQLite (with FTS5)	The database. One file, no server; foreign keys, checks, partial indexes, full-text search.

Software	Role
sqlite3 CLI	Running and inspecting SQL queries.
PostgreSQL	Considered in Phase 2 for comparison (range types and exclusion constraints).
Graphviz	Drawing the ER and architecture diagrams.
ReportLab	Printing the reports.
VS Code, Linux	Editor and operating system.

Table 3: Tools proposed for the project. Versions will be recorded when the program is run.

7. Preliminary entities

A first reading of the problem gives nine entity sets: agent, session, episode, entity, entity alias, fact, keyword, summary, and embedding. Figure 1 shows how they reference each other. Each arrow is a planned foreign key. Phase 2 turns this sketch into a full ER diagram with attributes, cardinalities, and constraints.

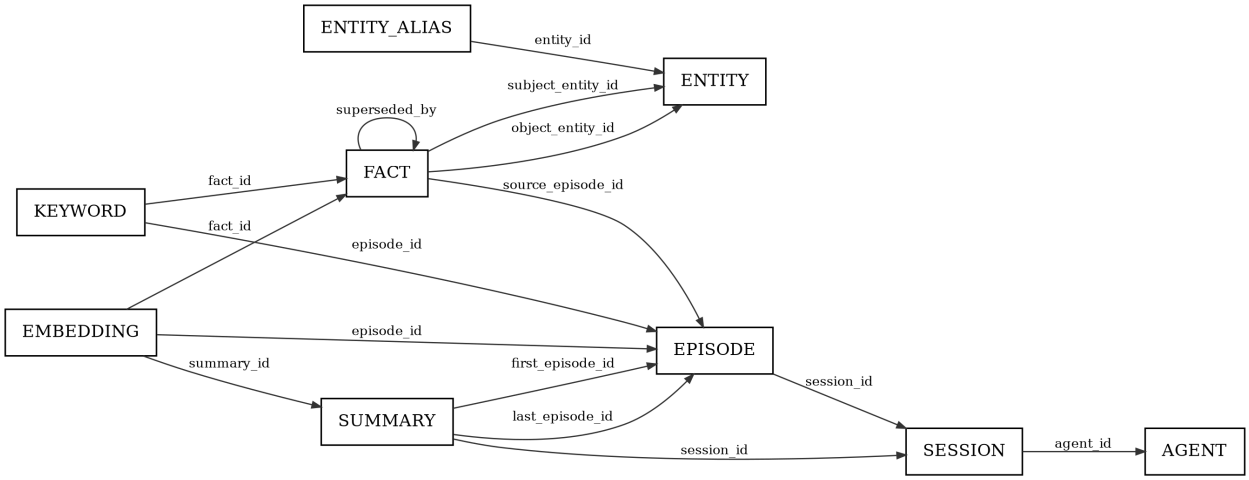


Figure 1: Preliminary entity sets. Each arrow points from the referencing table to the referenced table and is labelled with the planned foreign-key column.

8. Plan of work

Phase	Deliverable
Phase 1	This report: abstract, problem, motivation, literature review, objectives, scope, tools.
Phase 2	Survey of memory operations and substrates, requirements, ER diagram, relational schema, functional dependencies and normalisation, physical design plan.
Phase 3	Implementation in Python and SQLite, a scripted agent session, outputs, testing, and conclusions.

Table 4: The three phases of the project.

References

[1] Y. Du, W. Huang, D. Zheng, Z. Wang, S. Montella, M. Lapata, K.-F. Wong, and J. Z. Pan. *Rethinking Memory in AI: Taxonomy, Operations, Topics, and Future Directions*. arXiv:2505.00675, 2025.

[2] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez. *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560, 2023.

[3] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*. arXiv:2504.19413, 2025.

[4] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef. *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*. arXiv:2501.13956, 2025.

[5] Notion (makenotion). *lore: persistent, shared AI memory backed by Notion*. GitHub, MIT licence. https://github.com/makenotion/lore

[6] E. F. Codd. *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM 13(6):377-387, 1970.

[7] P. P.-S. Chen. *The Entity-Relationship Model: Toward a Unified View of Data*. ACM Transactions on Database Systems 1(1):9-36, 1976.

[8] K. Kulkarni and J.-E. Michels. *Temporal Features in SQL:2011*. ACM SIGMOD Record 41(3):34-43, 2012.

Thematic Assessment Report
Phase 2

TA REPORT PHASE 2

Persistent Memory Architecture in Agents Using DBMS

Survey, Requirements, and Database Design

Submitted in partial fulfilment of course requirements
for Database Management Systems

Bachelor of Technology
Computer Science and Engineering
School of Engineering and Technology

Submitted to:
Dr. Poonam Sangwan
Faculty, DBMS
Department of CSE / SOET
Date of submission: _____

Submitted by:
Harshit Khemani (241302081)
Kush Ahuja (241302122)
Kushagra Agrawal (241302109)
Madhav Bassi (241302100)
B.Tech CSE (AI/ML), Section C

https://github.com/HKTITAN/dbms-agent-memory

SGT University
School of Engineering and Technology

Declaration. We declare that this report is our own work for DBMS TA-2.

1. Introduction and objective

Phase 1 proposed a persistent memory for an AI agent, built as a relational database. It covered the problem, the objectives, the scope, and a short literature review. This phase does three things. It extends the survey to the memory operations an agent store must support and to four recent systems. It compares the substrates the same logical schema could run on: Notion databases (as used by Lore), SQLite, and PostgreSQL. Then it produces the database design: requirements, an ER diagram, the relational schema, the functional dependencies, and the normalisation argument.

The objective is a schema that Phase 3 can implement without changing it. Every table and column name in this report is the one the implementation uses.

2. Memory operations

Du and colleagues reduce memory in AI systems to six atomic operations [5]. Table 1 maps each operation to a database counterpart and to the operation our store will provide. In our design the operations are write, assert, supersede, forget, retrieve, and consolidate.

Operation [5]	Database counterpart	Planned operation in the store
Consolidation	Turning episodes into facts with keys and a validity interval	`assert`: insert a semantic fact that cites its episode
Indexing	Keyword rows, an FTS5 full-text index, an embedding per row	`write`: insert an episode and index it
Updating	Update without losing history: close the interval, insert a new row	`supersede`
Forgetting	A status flag, so the row and its foreign keys remain	`forget`
Retrieval	Query processing: predicate queries and ranked text search	`retrieve`: three ranked lists fused with RRF [6]
Compression	A materialised view: a stored query result	`consolidate`: one summary row per session

Table 1: The six memory operations of Du et al. [5] and their place in this design.

3. Related memory systems

Lore [1] is Notion's open-source memory system for agents. Memory is stored as pages in Notion databases, so a person can read and edit what the agent remembers. It ranks retrieved results with reciprocal rank fusion (RRF) [6], which combines several ranked lists by summing 1 / (k + rank). **Nemori** [2] organises memory the way cognitive science describes it. It segments conversation into episodes and uses a predict-calibrate step to decide what to keep as semantic memory. **LightMem** [3] is a lightweight memory pipeline that filters and groups input before storing it, and moves the expensive update offline into a sleep-time phase. **Harness the Memory** [4] evaluates memory substrates for memory agents and finds that no one substrate wins every task.

From these we take four decisions. Episodic and semantic memory are separate tables, as in Nemori. Retrieval fuses several ranked lists with RRF, as Lore does, with k = 60. A predicate query on current facts is kept beside the ranked text search, because no single access path wins every question [4]. Nemori's predict-calibrate loop and LightMem's sleep-time update are out of scope; consolidation stores the active facts of a session.

4. One logical schema on three substrates

The same logical schema can run on Notion databases (as in Lore), on SQLite, or on PostgreSQL. What changes is which rules the substrate itself enforces. Table 2 compares the three for the constraints this design needs.

Requirement	Lore (Notion databases)	SQLite	PostgreSQL
Readable and editable by people	Yes, as Notion pages	Through the sqlite3 shell	Through a client
Server or account needed	A Notion workspace	None: one file	A database server
Unique alias (one entity per alias)	Not declared; left to the application	PRIMARY KEY (alias_key)	PRIMARY KEY
Provenance (fact cites episode)	Not declared as a foreign key	FOREIGN KEY, enforced per connection	FOREIGN KEY
Domain and interval checks	Not declared	CHECK constraints	CHECK constraints
At most one current owner	Found by a scan afterwards	Partial unique index on active rows	Partial unique index
No two overlapping validity intervals	Not declared	Not expressible	EXCLUDE USING gist on a range
Full-text search	Notion's search endpoint	FTS5 virtual table	tsvector with a GIN index

Table 2: What each substrate can declare for the same schema.

Lore's vault stays readable, but it cannot declare these constraints. PostgreSQL can declare all of them, including the temporal exclusion constraint. SQLite can declare every one except that last one. It needs no server, so the program runs from a clone with no extra install. We choose SQLite for Phase 3. A partial unique index covers the current-owner rule, and the PostgreSQL exclusion constraint is noted as future work.

5. Requirements

5.1 Functional requirements

ID	Requirement
FR1	Record an agent, its sessions, and each turn of a session as an episode with a turn index and a role (user or assistant).
FR2	Register entities with a canonical name and aliases. Resolving an alias returns one entity. An alias cannot belong to two entities.
FR3	Assert a fact as subject, predicate, object, with a confidence, a stance, a valid-from time, and the episode it came from.
FR4	Refuse a second current object for the functional predicates `owned_by`, `is_a`, and `created_by`.
FR5	Supersede a fact: close its validity interval, insert the replacement, and link the old row to the new one.
FR6	Forget a fact or an episode without deleting it, so it is hidden from retrieval but provenance still joins.
FR7	Retrieve by keyword, by full-text search, and by vector similarity, and fuse the three rankings.
FR8	Answer what is believed now, and what was believed at a past time (an as-of query).
FR9	Consolidate a session into one stored summary of its active facts.
FR10	Refuse to delete an episode that a fact cites.

Table 3: Functional requirements.

5.2 Non-functional requirements

- Only the Python standard library and SQLite, so it needs no database server and no API key.
- Repeatable: the session uses fixed timestamps, so a second run prints the same output.
- Inspectable: every table, including stored vectors, can be read with the `sqlite3` shell.
- Every rule that can be declared in the schema is declared there, not only checked in Python, and `PRAGMA integrity_check` must pass.

6. Conceptual design

Table 4 lists the entity sets and their attributes. Primary keys are underlined. Episodic memory is `EPISODE`: one row per turn. Semantic memory is `FACT`: a triple with a confidence, a stance, and a validity interval. `ENTITY` is the registry of canonical names that facts point to.

Entity set	Attributes
AGENT	agent_id, name, created_at
SESSION	session_id, title, started_at
EPISODE	episode_id, turn_index, role, content, importance, forgotten, created_at, {keyword}
ENTITY	entity_id, name, name_key, kind, description, created_at, {alias}
FACT	fact_id, predicate, object_text, confidence, stance, valid_from, valid_until, status, statement, created_at, {keyword}
SUMMARY	summary_id, body, episode_count, created_at
EMBEDDING (weak)	embedding_id, dims, vector

Table 4: Entity sets and attributes.

Braces mark a multi-valued attribute. Table 5 lists the relationships.

Relationship	Between	Cardinality	Participation
opens	AGENT, SESSION	1 : N	Every session has an agent (total)
holds	SESSION, EPISODE	1 : N	Every episode belongs to a session (total)
yields	SESSION, SUMMARY	1 : N	Every summary belongs to a session (total)
supports	EPISODE, FACT	1 : N	A fact may cite its source episode (partial)
asserts	ENTITY, FACT	1 : N	Every fact has a subject entity (total)
refers to	ENTITY, FACT	1 : N	The object may be an entity or plain text (partial)
succeeds	FACT, FACT	1 : N	A superseded fact points to its replacement (partial)
embeds	EPISODE / FACT / SUMMARY, EMBEDDING	1 : N	Each embedding has exactly one owner (identifying)

Table 5: Relationships, cardinality, and participation.

Figure 1 is the ER diagram in Chen notation [8]. An entity is a rectangle, an attribute is an ellipse, and a relationship is a diamond. Primary keys are underlined, and cardinality is written on the lines. `alias` and `keyword` are multi-valued, so they are double ellipses. EMBEDDING depends on the episode, fact, or summary it describes, so it is a weak entity (double rectangle) and `embeds` is a double diamond. Only the main attributes are drawn; Table 4 has the full list.

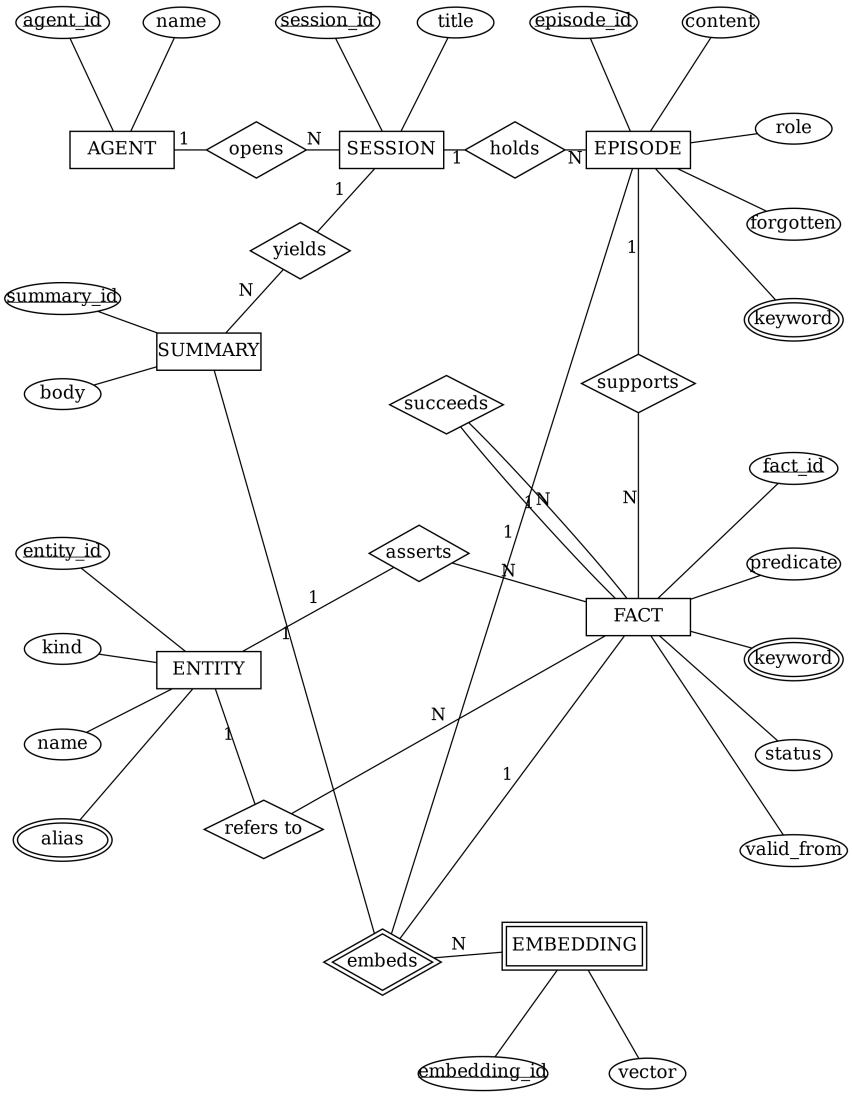


Figure 1: ER diagram of the memory store (Chen notation).

6.1 Constraints

- Domains: `role` is user or assistant; `kind` is person, team, system, project, or concept; `predicate` is is_a, uses, owned_by, created_by, or depends_on; `stance` is certain, likely, or speculative; `status` is active, superseded, or forgotten.
- Ranges: `confidence` in (0, 1]; `importance` in [0, 1]; `turn_index` >= 0; `episode_count` > 0; `dims` > 0.
- Valid time [9]: `valid_from` <= `valid_until`. An active fact has an open interval, and a superseded or forgotten fact has a closed one.
- A fact has an object: either an object entity or object text.
- Uniqueness: one turn index per session; one entity per `name_key`; one entity per alias; at most one active `owned_by`, `is_a`, or `created_by` per subject.
- A keyword belongs to exactly one episode or fact. An embedding belongs to exactly one episode, fact, or summary.

7. Logical design

Table 6 shows how each part of the ER model becomes a table, a column, or a constraint.

ER construct	Rule	Result
Strong entity sets	One table each, keyed by the entity key	agent, session, episode, entity, fact, summary
opens, holds, yields (1 : N)	Foreign key on the N side	session.agent_id, episode.session_id, summary.session_id
supports, asserts, refers to (1 : N)	Foreign keys on FACT	fact.source_episode_id, fact.subject_entity_id, fact.object_entity_id
succeeds (recursive 1 : N)	Self-referencing foreign key	fact.superseded_by references fact(fact_id)
Multi-valued alias	Own table, one row per value	entity_alias(alias_key PK, alias, entity_id FK)
Multi-valued keyword	Own table, one row per value	keyword(keyword_id, term, episode_id or fact_id), exactly one parent
Weak EMBEDDING, embeds	Table with a foreign key to each possible owner	embedding(episode_id, fact_id, summary_id), CHECK exactly one is set
Episode range of a summary	Two foreign keys to EPISODE	summary.first_episode_id, summary.last_episode_id

Table 6: ER-to-relational mapping.

The resulting relational schema [7] follows. It is printed from the catalogue of a database built from the schema file, with `PRAGMA table_info` and `PRAGMA foreign_key_list`. An arrow means a foreign key to that table.

```
agent(agent_id (PK), name, created_at)
session(session_id (PK), agent_id -> agent, title, started_at)
episode(episode_id (PK), session_id -> session, turn_index, role, content, importance,
    forgotten, created_at)
entity(entity_id (PK), name, name_key, kind, description, created_at)
entity_alias(alias_key (PK), alias, entity_id -> entity)
fact(fact_id (PK), subject_entity_id -> entity, predicate, object_text, object_entity_id ->
    entity, confidence, stance, valid_from, valid_until, status, source_episode_id -> episode,
    superseded_by -> fact, statement, created_at)
keyword(keyword_id (PK), term, episode_id -> episode, fact_id -> fact)
summary(summary_id (PK), session_id -> session, body, episode_count, first_episode_id ->
    episode, last_episode_id -> episode, created_at)
embedding(embedding_id (PK), episode_id -> episode, fact_id -> fact, summary_id -> summary,
    dims, vector)
```

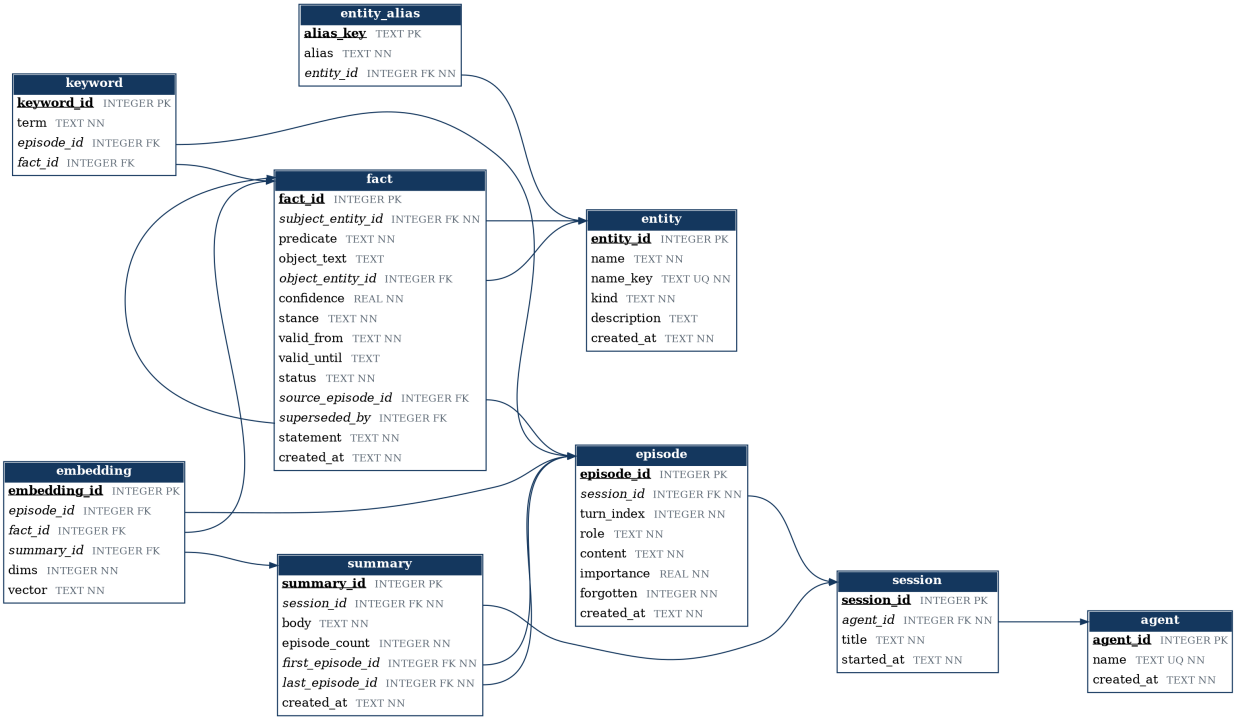


Figure 2: Relational schema. Primary keys are underlined, foreign keys are in italics, and each arrow is a foreign key. UQ is unique and NN is NOT NULL.

8. Functional dependencies and normalisation

8.1 Starting point

A memory written as notes, without a design, puts everything about one turn into one row. Table 7 shows two such rows from the worked example used throughout the project: the agent Archivist records a design session for Campus Navigator, in which ownership moves from the DBMS project team to the AI Lab.

Attribute	Row for turn 1	Row for turn 5
`agent`	Archivist	Archivist
`session_title`	Campus Navigator design	Campus Navigator design
`turn_index`	1	5
`content`	Campus Navigator is owned by the DBMS project team.	Ownership moved. The AI Lab now owns Campus Navigator.
`subject`	Campus Navigator	Campus Navigator
`subject_aliases`	Campus Navigator, navigator, the app	Campus Navigator, navigator, the app
`predicate`	owned_by	owned_by
`object`	DBMS project team	AI Lab
`valid_from`	2026-09-30T09:10:00	2026-09-30T09:40:00
`valid_until`	2026-09-30T09:40:00	NULL (open)
`keywords`	campu, navigator, own, dbm, project, team	ownership, mov, ai, lab, now, own, campu, navigator

Table 7: Two unnormalised MEMORY_NOTE rows, shown one per column. subject_aliases and keywords are repeating groups.

8.2 First normal form

The subject_aliases cell and the keywords cell each hold a list, so MEMORY_NOTE is not in 1NF. Each list becomes rows of its own table: `entity_alias(alias_key, alias, entity_id)`, one row per alias, and `keyword(keyword_id, term, episode_id, fact_id)`, one row per term. A query for "the app" is then an equality on a key, not a substring search inside a cell.

8.3 Second normal form

After 1NF the turn is identified by the composite key `(session_id, turn_index)`. The session title and the agent depend on the session alone, not on the whole key. That is a partial dependency, so it moves to `session(session_id, agent_id, title, started_at)`. The turn becomes `episode` with the surrogate key `episode_id`, and `UNIQUE (session_id, turn_index)` is kept as a candidate key.

8.4 Third normal form

Two transitive dependencies remain. The agent's name depends on the agent, which depends on the session, so it moves to `agent`. The subject's name and kind depend on the entity, not on the turn, so they move to `entity`, and the fact refers to it by `subject_entity_id`. A fact also has its own life: one turn may state no fact or several, and a fact's validity interval changes when it is superseded. So facts become their own table, `fact`, which points back to the turn through `source_episode_id`. That foreign key is the provenance link.

8.5 Boyce-Codd normal form

Table 8 lists the functional dependencies of each table. In every table, each determinant is a superkey.

Table	Functional dependencies	Candidate keys	BCNF
agent	agent_id → name, created_at; name → agent_id	{agent_id}, {name}	Yes
session	session_id → agent_id, title, started_at	{session_id}	Yes
episode	episode_id → all; (session_id, turn_index) → all	{episode_id}, {session_id, turn_index}	Yes
entity	entity_id → all; name_key → all	{entity_id}, {name_key}	Yes
entity_alias	alias_key → alias, entity_id	{alias_key}	Yes
fact	fact_id → all other attributes	{fact_id}	Yes, except the derived statement (8.6)
keyword	keyword_id → term, episode_id, fact_id	{keyword_id}	Yes
summary	summary_id → all	{summary_id}	Yes
embedding	embedding_id → all	{embedding_id}	Yes

Table 8: Functional dependencies and BCNF check.

8.6 Derived columns kept on purpose

- `entity.name_key` and `entity_alias.alias_key` are the case-folded name and alias, so "AI Lab" and "ai lab" cannot become two rows. They are derived, and they are what the uniqueness is declared on.
- `fact.statement` is the sentence made from the subject name, the predicate, and the object. It is determined by the triple, which is not a key, so strictly it breaks BCNF. It is kept because the full-text index reads its text from a column of the base table.

- `summary.body` and `episode_count` are a snapshot taken at consolidation. They are a stored query result (a materialised view), not a second copy that has to stay in step.
- `embedding.vector` is computed from the row's text and stored as one value. The database never queries a single element of it, so it is treated as atomic.

9. Physical design plan

Table 9 lists the indexes and other physical structures the schema declares. Besides these, the views `belief_history` (every fact with subject and object names) and `current_belief` (active facts only) give the current-belief and as-of queries a stable shape.

Structure	On	Purpose
`ux_one_active_functional_fact`	fact (subject_entity_id, predicate)	Partial unique index: at most one active owned_by, is_a, or created_by per subject (FR4).
`ix_fact_source`	fact (source_episode_id)	Provenance joins from a fact to its episode.
`ix_fact_validity`	fact (valid_from, valid_until)	As-of queries on the validity interval (FR8).
`ix_fact_object`	fact (object_entity_id)	Facts that refer to an entity as the object.
`ix_keyword_term`	keyword (term)	Keyword lookup (FR7).
`episode_fts`	episode (content)	FTS5 full-text index, external content, porter tokenizer; kept in step by triggers.
`fact_fts`	fact (statement)	FTS5 full-text index over fact statements; kept in step by triggers.

Table 9: Indexes and full-text structures, read from the schema catalogue.

The rule behind FR4 is declared as a partial unique index:

```
-- At most one *current* object for predicates that are functions of the
-- subject. This is the constraint a conflict scan otherwise has to rediscover.
CREATE UNIQUE INDEX ux_one_active_functional_fact
  ON fact (subject_entity_id, predicate)
  WHERE status = 'active'
    AND predicate IN ('owned_by', 'is_a', 'created_by');
```

A vector is planned as a signed feature hash of the row's tokens into 512 buckets [10], stored as a JSON array. That needs no model and no network call. A real embedding model can replace it later without changing the schema. The as-of query planned for FR8 reads the history view:

```
SELECT subject, object, status, valid_from, valid_until
FROM belief_history
WHERE subject = 'Campus Navigator'
  AND predicate = 'owned_by'
  AND valid_from <= '2026-09-30T09:30:00'
  AND (valid_until IS NULL OR valid_until > '2026-09-30T09:30:00');
```

The other planned queries list current beliefs, list aliases one row per alias, join each active fact to the episode that supports it, list the foreign keys on `fact`, and run `PRAGMA integrity_check`.

10. Recommendation and plan for Phase 3

The recommendation is a local relational store with episodic rows, semantic triples with a validity interval, a normalised entity and alias table, provenance back to the episode that supported a claim, and indexes for keyword and vector retrieval. Phase 3 implements this schema in SQLite as `schema.sql`, with a Python

module for write, assert, supersede, forget, retrieve, and consolidate, and a scripted agent session that exercises every requirement in Table 3 and prints checks.

References

[1] Notion (makenotion). *lore: persistent, shared AI memory backed by Notion*. GitHub, MIT licence, commit 95c3558. https://github.com/makenotion/lore

[2] W. Ma, J. Nan, W. Wu, and Y. Chen. *Nemori: Self-Organizing Agent Memory Inspired by Cognitive Science*. arXiv:2508.03341, 2025.

[3] J. Fang, X. Deng, H. Chen, N. Zhang, et al. *LightMem: Lightweight and Efficient Memory-Augmented Generation*. ICLR 2026. arXiv:2510.18866.

[4] W.-C. Huang, W. Zhang, Y. Wu, Y. Chen, et al. *Harness the Memory: A Holistic Evaluation of Memory Substrates in Memory Agents*. arXiv:2608.15008, 2026.

[5] Y. Du, W. Huang, D. Zheng, Z. Wang, S. Montella, M. Lapata, K.-F. Wong, and J. Z. Pan. *Rethinking Memory in AI: Taxonomy, Operations, Topics, and Future Directions*. arXiv:2505.00675, 2025.

[6] G. V. Cormack, C. L. A. Clarke, and S. Buettcher. *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. SIGIR 2009.

[7] E. F. Codd. *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM 13(6):377-387, 1970.

[8] P. P.-S. Chen. *The Entity-Relationship Model: Toward a Unified View of Data*. ACM Transactions on Database Systems 1(1):9-36, 1976.

[9] K. Kulkarni and J.-E. Michels. *Temporal Features in SQL:2011*. ACM SIGMOD Record 41(3):34-43, 2012.

[10] K. Weinberger, A. Dasgupta, J. Langford, A. Smola, and J. Attenberg. *Feature Hashing for Large Scale Multitask Learning*. ICML 2009.

Thematic Assessment Report
Phase 3

TA REPORT PHASE 3

Persistent Memory Architecture in Agents Using DBMS

Implementation (Python · SQLite · FTS5)

Submitted in partial fulfilment of course requirements
for Database Management Systems

Bachelor of Technology
Computer Science and Engineering
School of Engineering and Technology

Submitted to:
Dr. Poonam Sangwan
Faculty, DBMS
Department of CSE / SOET
Date of submission: 1 October 2026

Submitted by:
Harshit Khemani (241302081)
Kush Ahuja (241302122)
Kushagra Agrawal (241302109)
Madhav Bassi (241302100)
B.Tech CSE (AI/ML), Section C

https://github.com/HKTITAN/dbms-agent-memory

SGT University
School of Engineering and Technology

Declaration. We declare that this report and the program in `ta3-implementation/` are our own work for DBMS TA-3.

1. Introduction and objective

Phase 1 of this term paper reviewed persistent memory for AI agents. Phase 2 compared Notion's Lore with the same logical schema on SQLite and PostgreSQL, and added Nemori, LightMem, and *Harness the Memory*. The recommendation was a local relational store: episodic rows, semantic triples with a validity interval, a normalised entity and alias table, provenance back to the episode that supported a claim, and indexes for keyword and vector retrieval. Lore's vault stays readable, but it cannot declare those constraints. This phase implements that store.

The objective is a small program a reader can run. It has to support write, retrieve (keyword and vector), consolidate, update, and forget, on a schema with keys and checks, and print the result of one agent session.

2. Software and tools

The memory store imports only the Python standard library. The versions below were read on the machine that produced the screenshots.

Software	Version	Role
Python	3.12.3	`sqlite3` is in the standard library, so `python3 demo.py` needs no extra packages.
SQLite	3.45.1	One file, with foreign keys, checks, a partial unique index, and FTS5. No server.
sqlite3 CLI	3.45.1	Runs `queries.sql` at the end of the demo.
VS Code	1.139.1	Editor used for the code screenshots.
xfce4-terminal	xfce4-terminal 1.1.3 (Xfce 4.18)	Terminal used to run `python3 demo.py`.
Graphviz	2.43.0 (0)	`dot` and `neato` drew the diagrams.
ReportLab	5.0.1	Prints this PDF. Not used by the demo.
Linux	Ubuntu 24.04.4 LTS; kernel 6.12.94+	The machine the demo was run on (x86_64).

PostgreSQL would add a temporal exclusion constraint, which SQLite cannot express. That is noted under future work. SQLite is enough to show primary keys, foreign keys, checks, and a partial unique index, and the program runs from a clone with no extra install.

3. System architecture

An agent session does not talk to SQL itself. `demo.py` calls `MemoryStore`. The class opens one SQLite connection, turns foreign keys on (SQLite leaves them off unless asked), and applies `schema.sql`. Each operation is a short transaction. Retrieval is three queries whose ranks are fused in Python with reciprocal rank fusion, the same rule Phase 2 noted in Lore, with the constant k = 60.

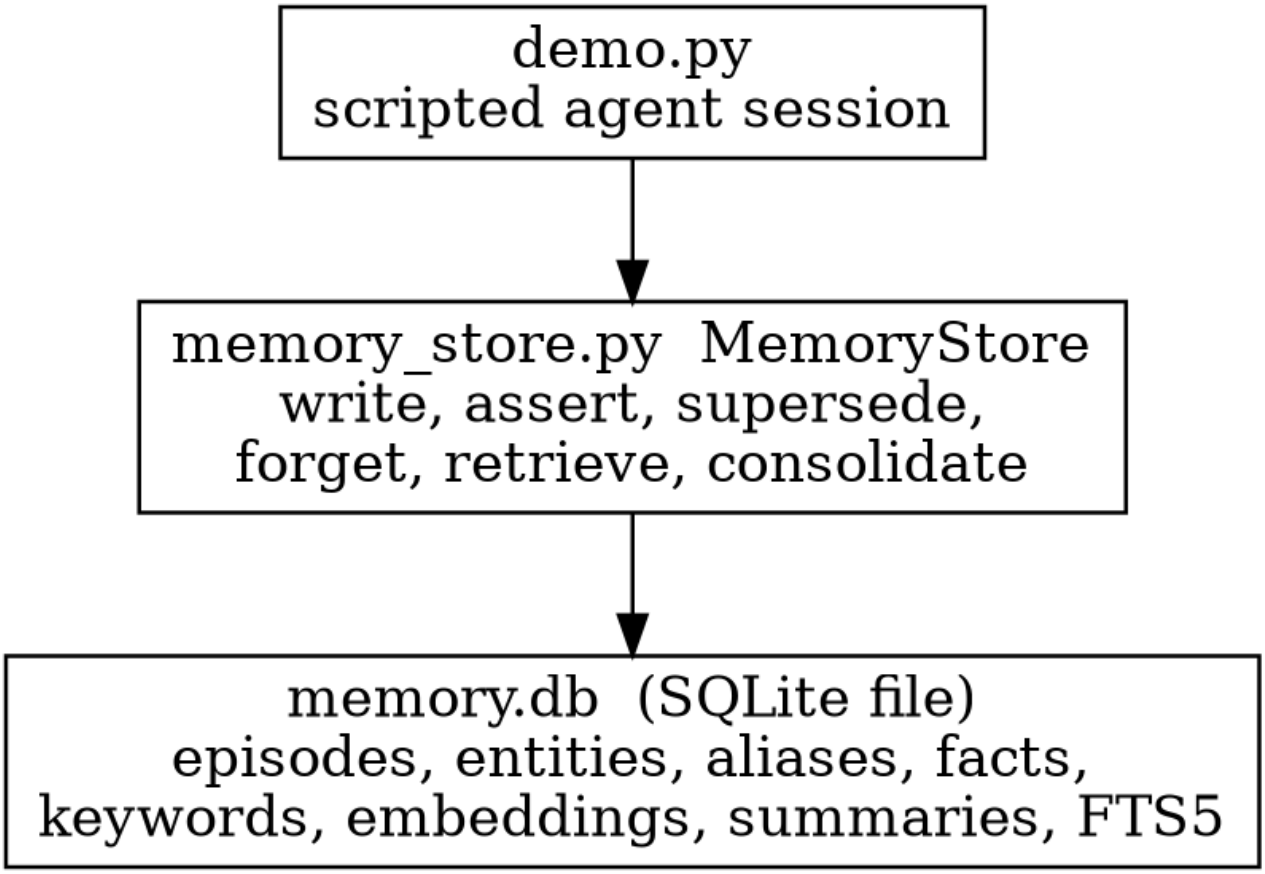


Figure 1: How the pieces fit. `demo.py` *calls* `MemoryStore`*, and that module is the only writer of* `memory.db`*.*

The six memory operations from the Phase 2 survey map onto the schema as follows. Write inserts an episode and indexes it. Assert inserts a semantic fact. Supersede is the update: the old row's interval is closed and a new row is inserted. Forget changes a status flag and keeps the row, so provenance still joins. Retrieve reads. Consolidate inserts a summary row. That summary is the active facts from the session, not a second copy of the episodes.

4. Database schema

Figure 2 is the ER diagram of the memory store, drawn in Chen notation. An entity is a rectangle, an attribute is an ellipse, and a relationship is a diamond. Primary keys are underlined. Cardinality (1 or N) is written on the lines. `alias` and `keyword` are multi-valued, so they are double ellipses. Embedding depends on the episode, fact, or summary it describes, so it is a weak entity (double rectangle) and {code('embeds')} is a double diamond. A later fact that replaces an earlier one is the {code('succeeds')} relationship. In the tables, a multi-valued attribute is its own rows: `alias_key` is a primary key, and `keyword` has a check that exactly one parent is set.

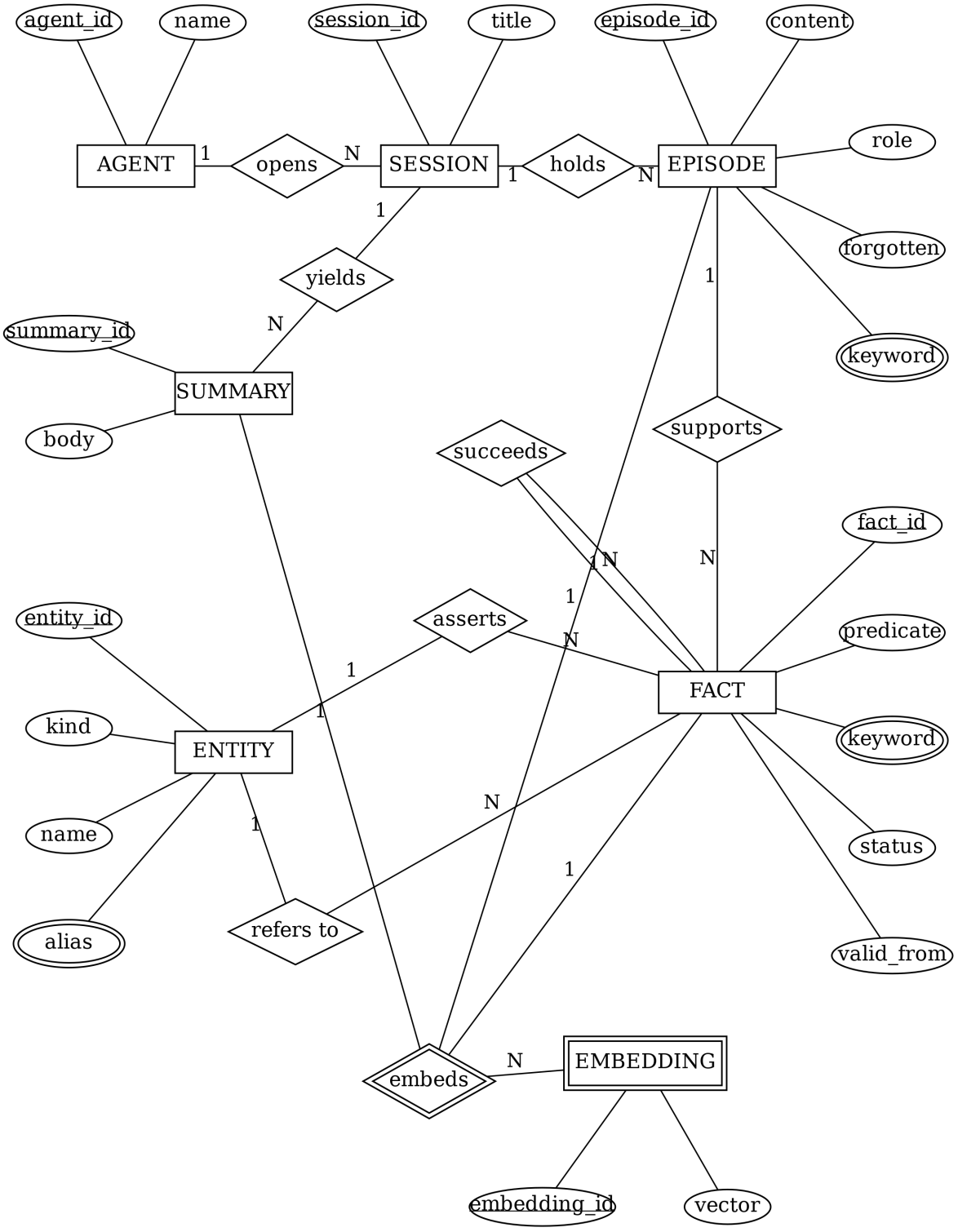


Figure 2: ER diagram of the memory store.

`FACT` is the semantic memory. `valid_from` and `valid_until` are valid time. A check requires an active fact to have an open interval, and a superseded or forgotten fact to have a closed one. The partial unique index `ux_one_active_functional_fact` allows many historical owners and at most one active `owned_by`, `is_a`, or `created_by` per subject. `uses` is not in that index: an application may use

more than one system, and a wrong guess is forgotten rather than blocked. Views `belief_history` and `current_belief` are what the as-of query and the current-owner query read.

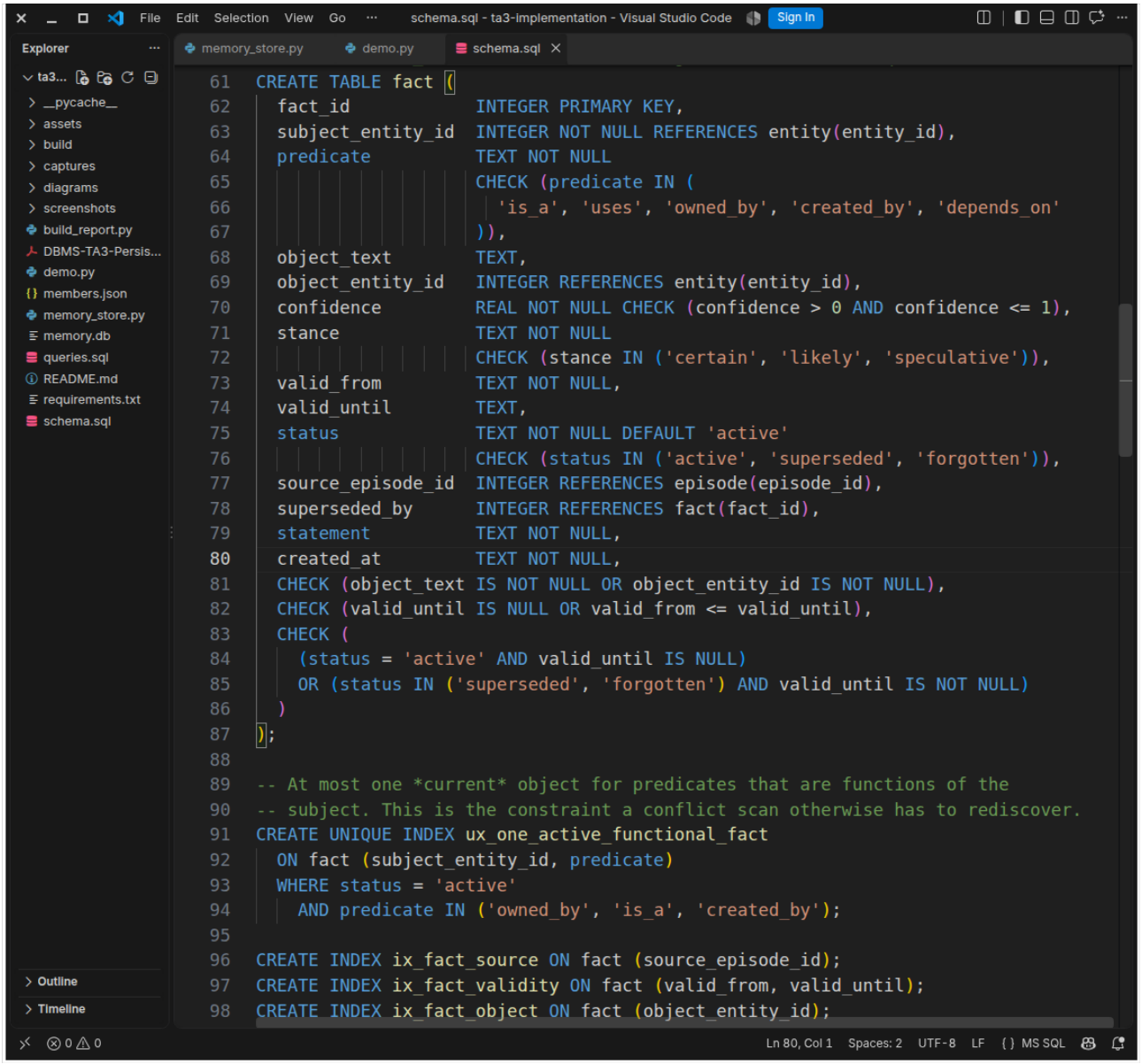


Figure 3: The fact table and the partial unique index in `schema.sql`, open in VS Code. An active fact must have an open interval. The index allows many past owners and only one current `owned_by`, `is_a`, or `created_by`.

5. Implementation

`memory_store.py` is the only module. Tokens are lower-cased, stopwords are dropped, and a short suffix stem (owns and owned both become own) makes the keyword table agree with the FTS5 porter tokenizer. Embeddings are a signed feature hash into 512 buckets, using SHA-256 so the vector does not change between processes. Cosine similarity is the dot product of two unit vectors. There is no neural model and no network call. The embedding table stores a JSON array of floats, which `sqlite3` can print.

`assert_fact` resolves names through `entity_alias`, so the session can say "the app" and hit Campus Navigator. If a functional predicate already has a different active object, the method raises `FactConflict` and does not insert. `supersede_fact` then closes the old interval and inserts the replacement in one transaction: it marks the old row superseded, inserts the new row, and sets

`superseded_by`. The unique index is never asked to hold two active owners. Figure 8 shows what a raw second insert does.

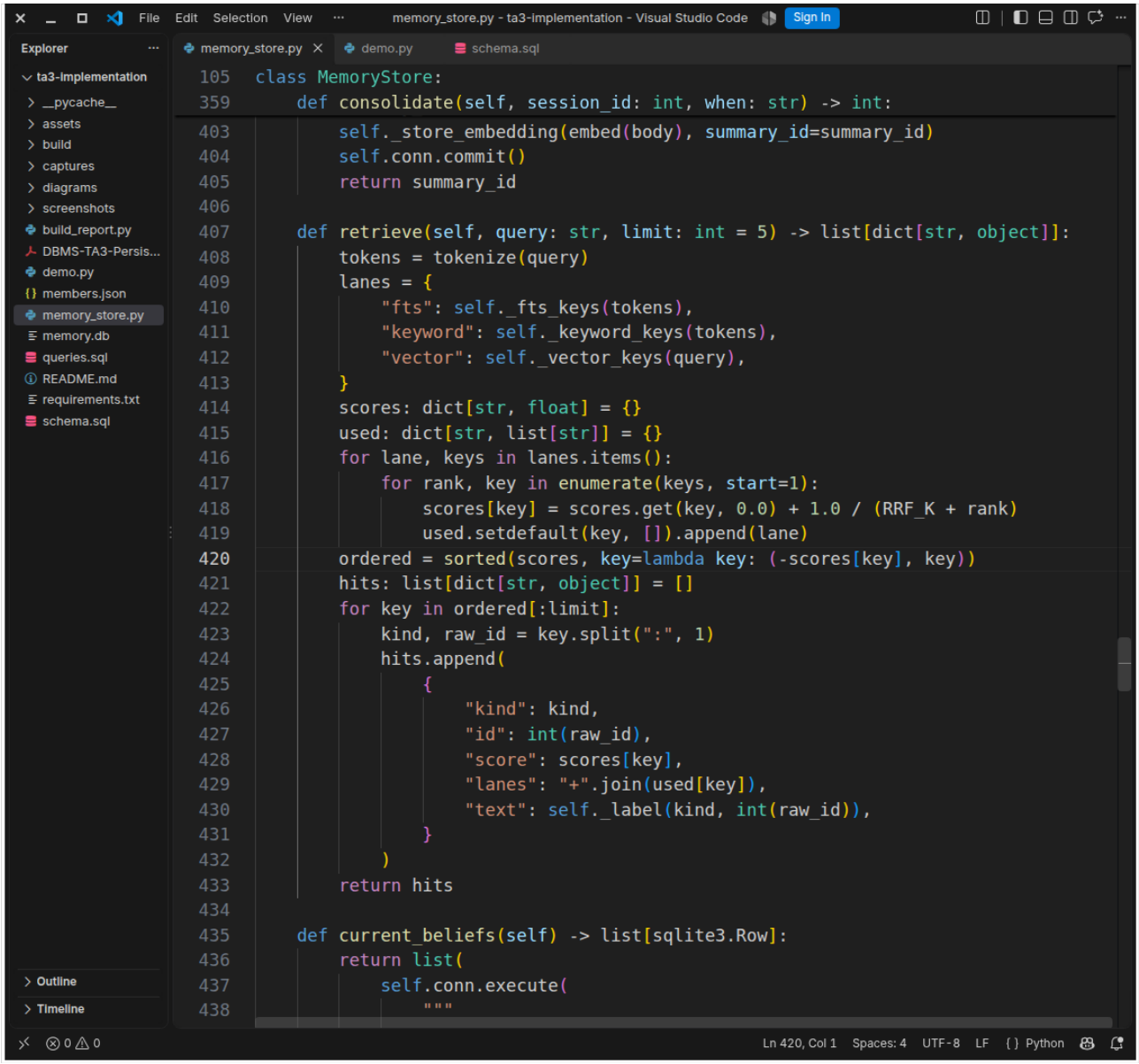


Figure 4: `retrieve` *in* `memory_store.py`. *Three ranked lists are fused with reciprocal rank fusion, k = 60. The lanes field is what the transcript prints.*

`forget_fact` sets status to forgotten and writes `valid_until`. `forget_episode` sets forgotten and deletes that episode's keyword rows. The episode row stays, because a fact may still cite it. Deleting the episode outright is refused by the foreign key, which the demo provokes and then rolls back. `consolidate` inserts one summary row whose text is the active facts sourced from the session, with their confidences. That is a stored row, not a language-model summary.

`demo.py` is the session. The clock is a list of fixed timestamps, so a second run prints the same transcript. Figure 5 is the part that registers entities and the six turns.

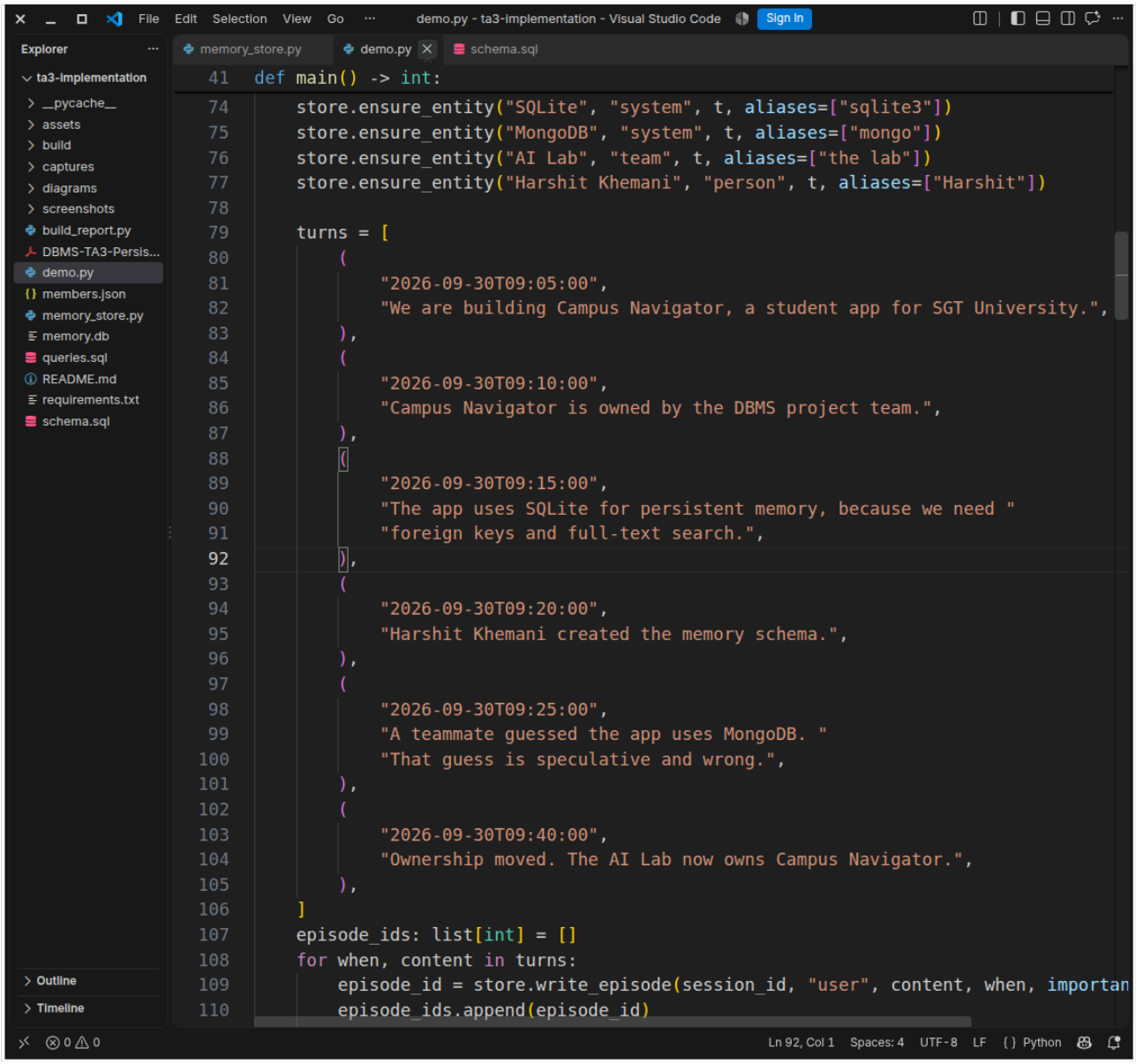


Figure 5: The scripted session in `demo.py`. *Aliases are registered with the entity, and each turn is one episode.*

6. Output and results

`python3 demo.py` deletes any existing `memory.db`, applies the schema, and runs the session. The figures in this section are screenshots of that run in xfce4-terminal. The same transcript is saved in `captures/demo-stdout.txt` when this PDF is rebuilt.

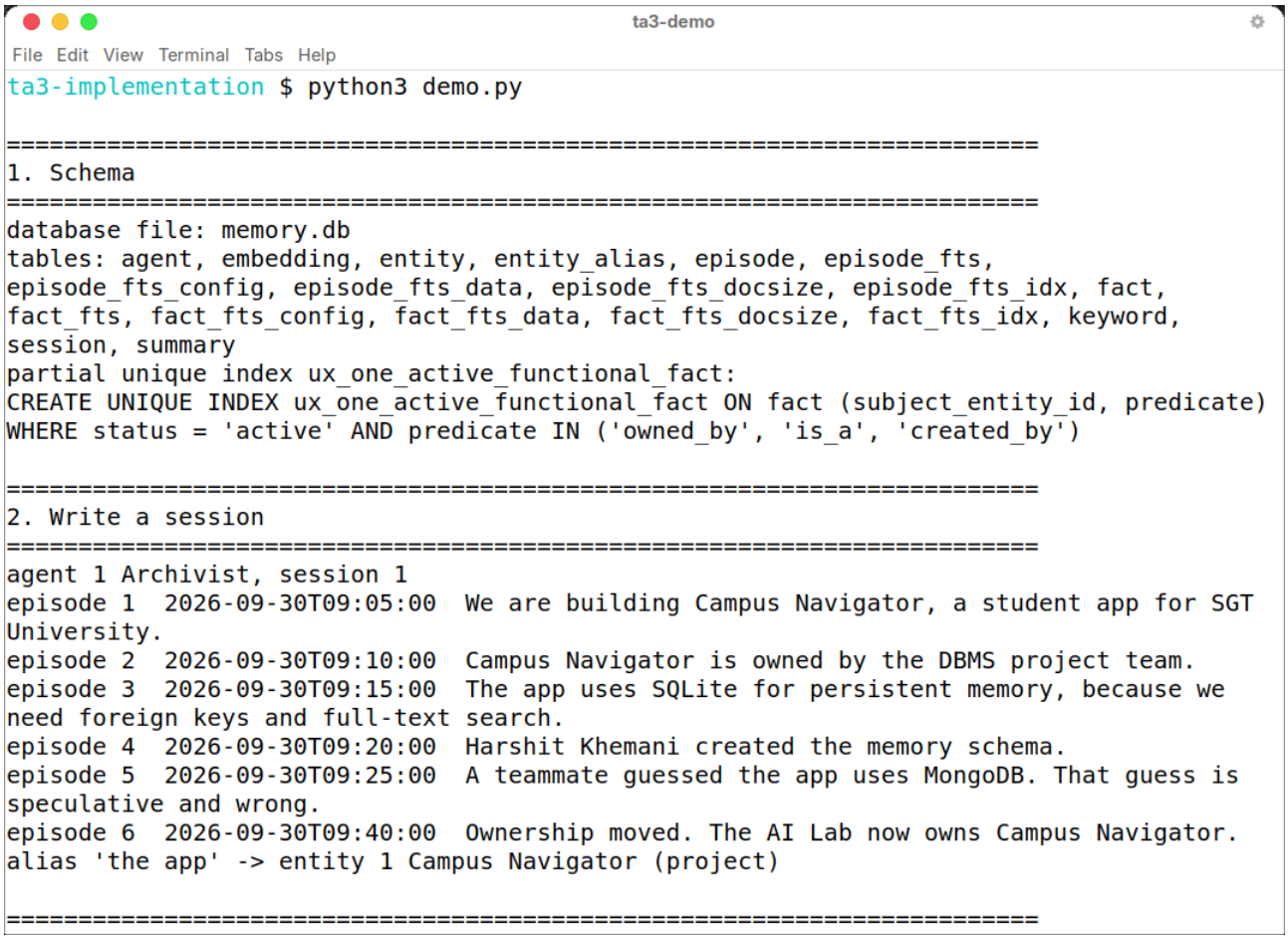


Figure 6: The start of `python3 demo.py`. *The partial unique index is printed from* `sqlite_master`, *then the six episodes. The alias* `the app` *resolves to entity 1, Campus Navigator.*

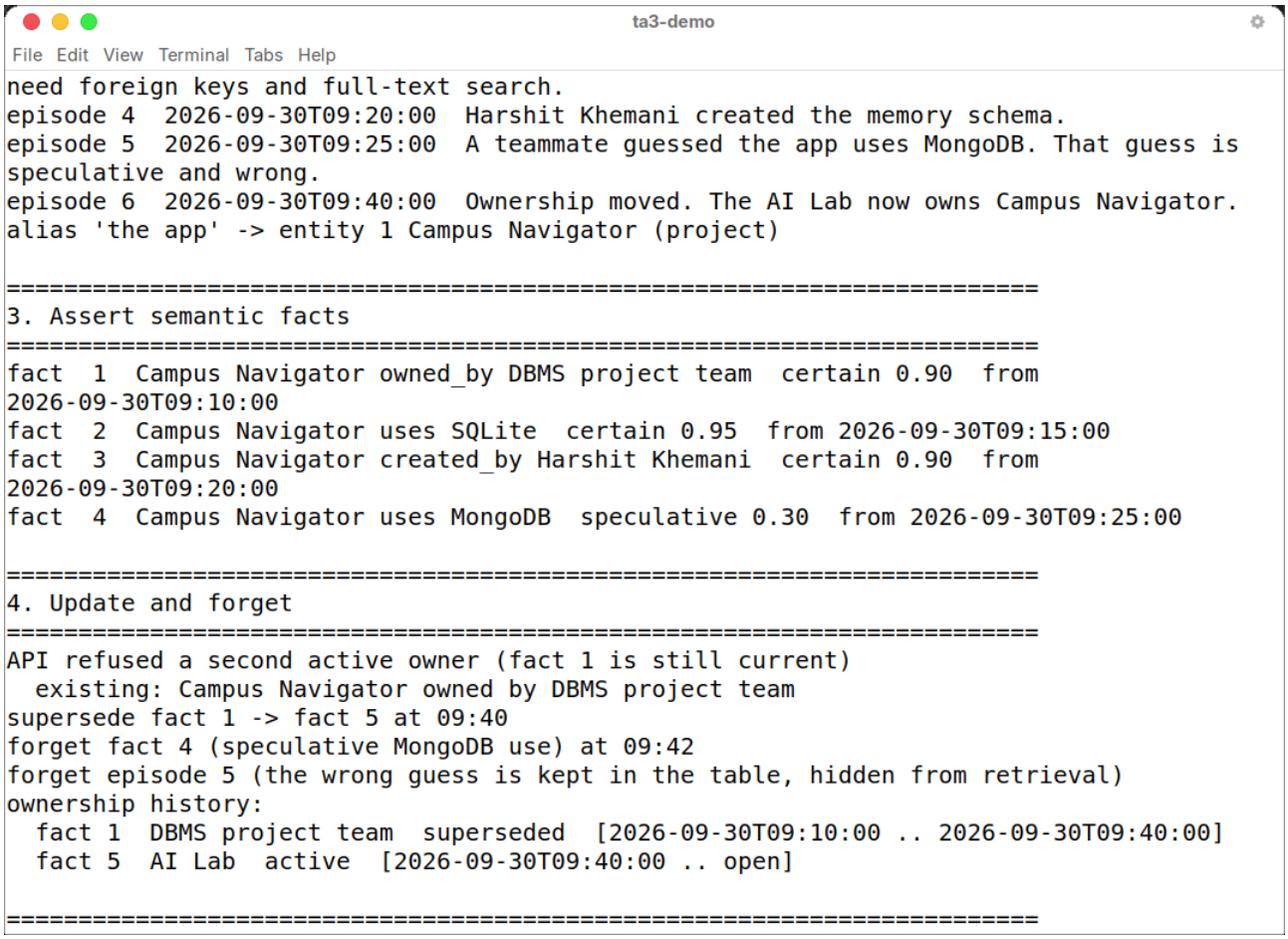


Figure 7: Facts, then the update. The API refuses a second active owner. Supersede closes fact 1 at 09:40 and inserts fact 5. Fact 4 and episode 5 (the MongoDB guess) are forgotten. The history shows fact 1 superseded and fact 5 active.

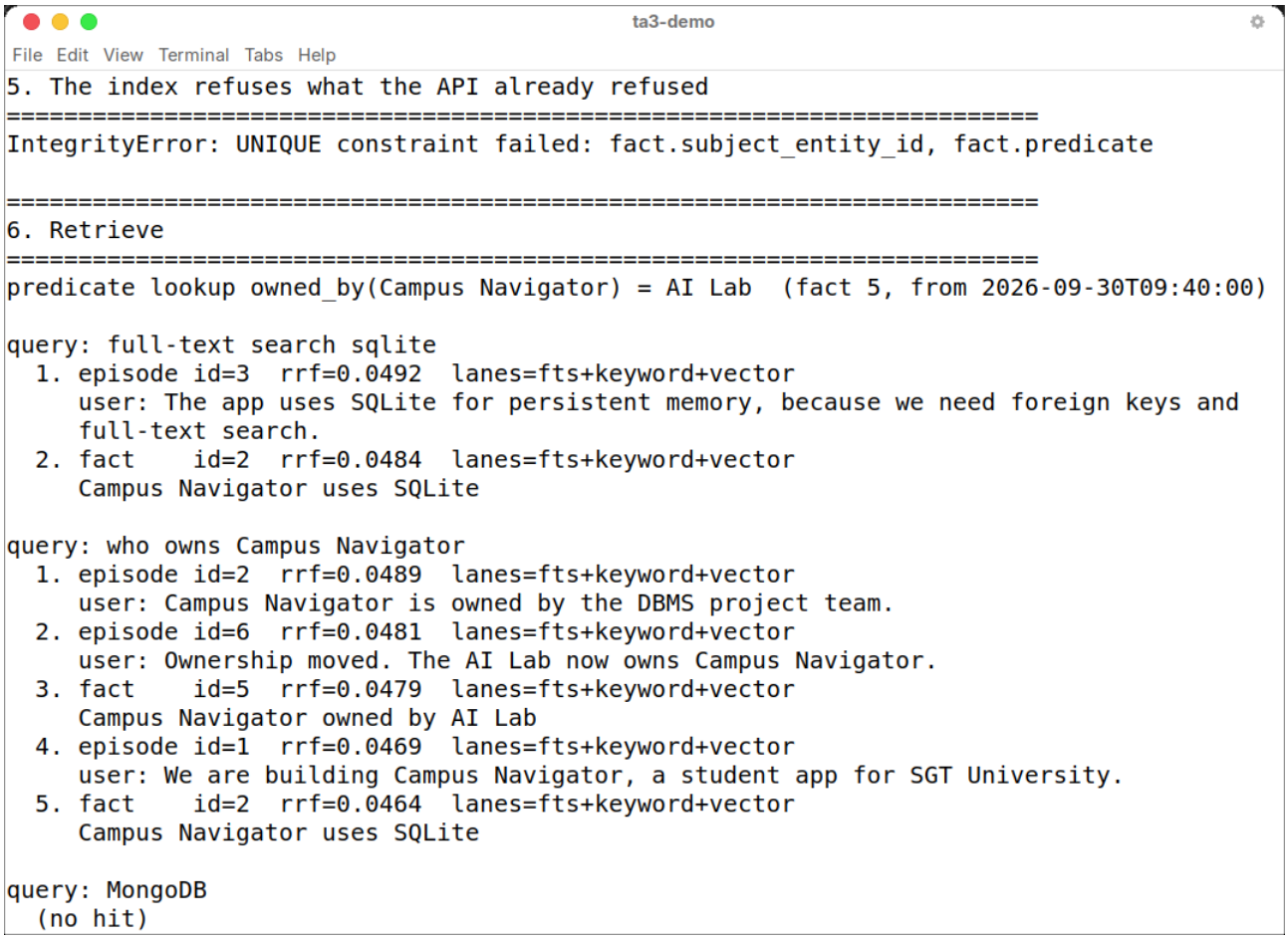


Figure 8: A raw INSERT of another active `owned_by` *fails with UNIQUE constraint failed. The predicate query returns AI Lab, fact 5. Lexical fusion ranks the old ownership episode first and the current fact third. After the MongoDB rows are forgotten, the query MongoDB returns no hit.*

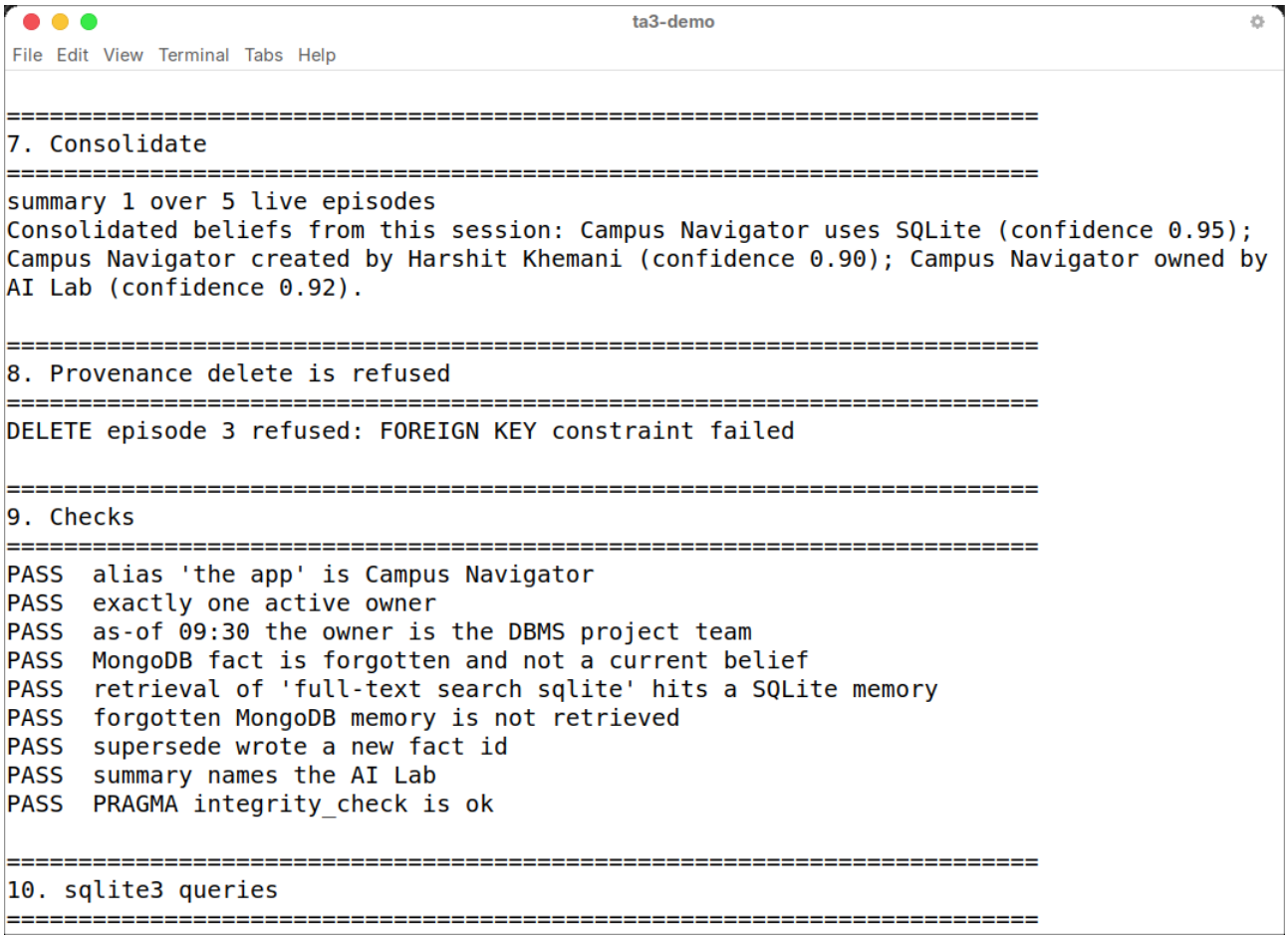


Figure 9: Consolidation, the foreign-key refusal, and the nine checks. The summary names SQLite, Harshit Khemani, and AI Lab, and does not name MongoDB. Deleting episode 3, which fact 2 cites, is refused.

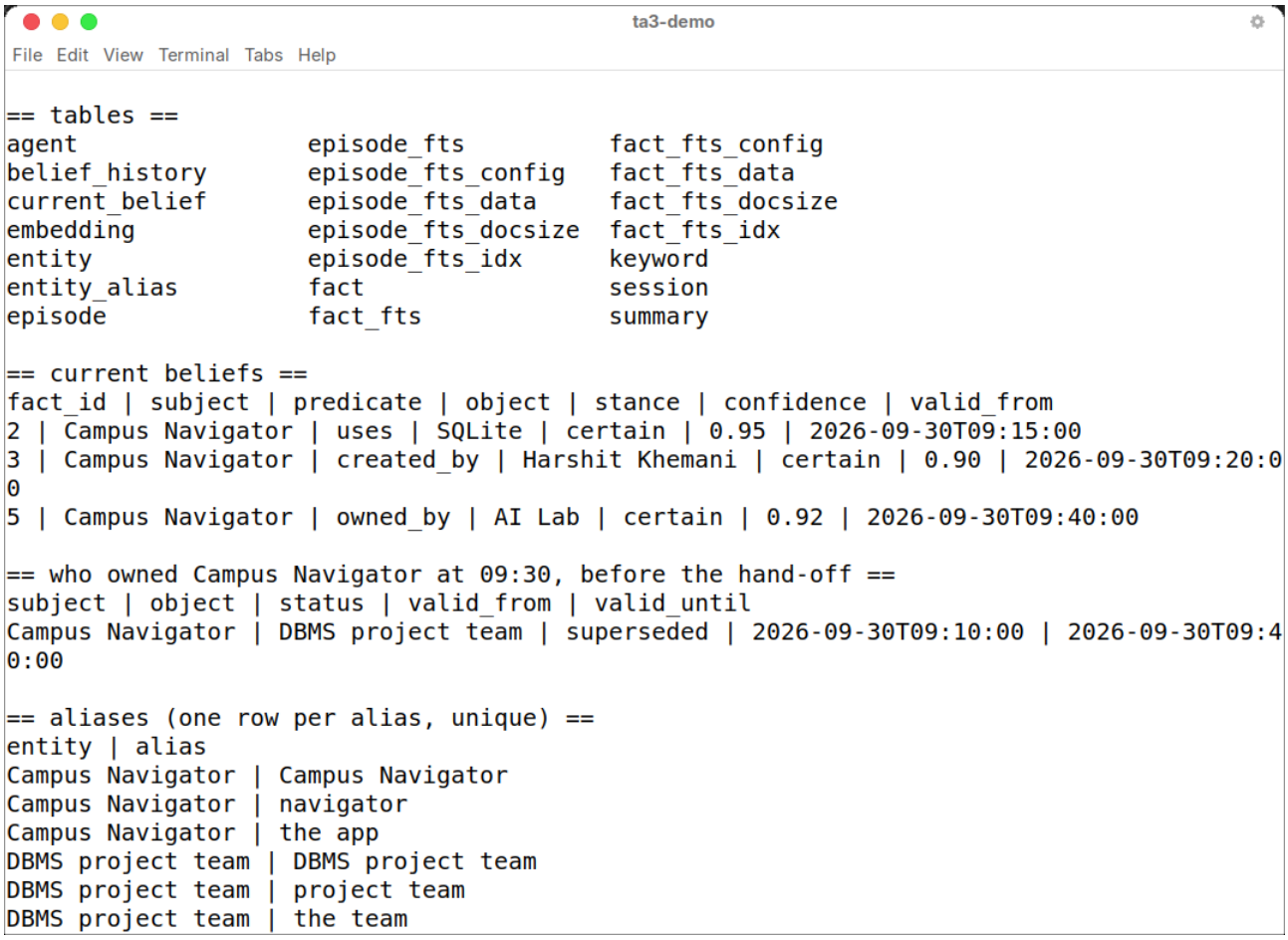


Figure 10: The sqlite3 section of the same run. Three facts are current. At 09:30 the owner was the DBMS project team. Aliases are one row each.

7. Testing and observations

The demo ends in nine checks, all passed on the run in the figures: alias resolution, a single active owner, the 09:30 as-of result, the forgotten MongoDB fact absent from `current_belief`, a SQLite hit for "full-text search sqlite", an empty result for "MongoDB", a new fact id from supersede, the summary naming AI Lab, and `PRAGMA integrity_check`. The process exits 0 only if every check passes. Two consecutive runs printed the same transcript.

Two results are easy to mix up. First, lexical retrieval of "who owns Campus Navigator" ranks episode 2 (the DBMS project team) above fact 5 (AI Lab). The old sentence is still in the episode table, and it shares the query's words. The question "who owns it now" is the predicate query on `current_belief`, which returns AI Lab only. The as-of query is what returns the earlier owner. Keyword search and the validity interval answer different questions, and the transcript shows both. Second, forgetting removes the MongoDB rows from retrieval without deleting the entity. The entity and its alias remain, which is what a registry is for.

The foreign-key refusal in Figure 9 happens on the Python connection, which has executed `PRAGMA foreign_keys = ON`. A bare `sqlite3` shell does not turn that pragma on by itself. The keys are declared either way. Enforcement is per connection. `queries.sql` still prints `PRAGMA foreign_key_list(fact)`, and that listing has the four foreign keys on `fact`.

8. Limitations and future work

The feature hash measures token overlap. It is not a semantic embedding, and similarity is a scan of a few dozen vectors rather than an approximate index. A model can replace embed without a schema change: the column is already a vector of floats. Consolidation does not implement Nemori's predict-calibrate loop or LightMem's sleep-time update. It stores the active facts of one session. The partial unique index is weaker than the PostgreSQL exclusion constraint in Phase 2: it limits active rows, and it does not reject two closed intervals that overlap. The demo is a single writer, so it does not reproduce a lost update. The unique index is what would make that race fail instead of silently keeping both rows. There is no authentication.

Harness the Memory found that no one substrate wins every task. This program already runs a predicate query beside the fused lexical lanes. A later version can route a question to one or the other instead of always printing both. Moving the same schema to PostgreSQL would be the place to add the range exclusion constraint SQLite does not have.

9. Conclusion

TA-3 is a runnable SQLite memory for one agent. Episodes, entities, aliases, facts, keywords, embeddings, and a session summary are ordinary tables with keys and checks. The session writes six turns, refuses a second current owner until the old interval is closed, forgets a speculative fact without losing the audit trail, and answers "who owns Campus Navigator" both as a predicate query (AI Lab, from 09:40) and as a fused retrieval over the text. Nine checks pass from a clean database file.

References

[1] Notion (makenotion). *lore: persistent, shared AI memory backed by Notion*. GitHub, MIT licence, commit 95c3558. https://github.com/makenotion/lore

[2] W. Ma, J. Nan, W. Wu, and Y. Chen. *Nemori: Self-Organizing Agent Memory Inspired by Cognitive Science*. arXiv:2508.03341, 2025. Revised as *What Deserves Memory: Adaptive Memory Distillation for LLM Agents*.

[3] J. Fang, X. Deng, H. Chen, N. Zhang, et al. *LightMem: Lightweight and Efficient Memory-Augmented Generation*. ICLR 2026. arXiv:2510.18866.

[4] W.-C. Huang, W. Zhang, Y. Wu, Y. Chen, et al. *Harness the Memory: A Holistic Evaluation of Memory Substrates in Memory Agents*. arXiv:2608.15008, 2026.

[5] Y. Du, W. Huang, D. Zheng, Z. Wang, S. Montella, M. Lapata, K.-F. Wong, and J. Z. Pan. *Rethinking Memory in AI: Taxonomy, Operations, Topics, and Future Directions*. arXiv:2505.00675, 2025.

[6] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez. *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560, 2023.

[7] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*. arXiv:2504.19413, 2025.

[8] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef. *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*. arXiv:2501.13956, 2025.

[9] G. V. Cormack, C. L. A. Clarke, and S. Buettcher. *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. SIGIR 2009.

[10] E. F. Codd. *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM 13(6):377-387, 1970.

[11] P. P.-S. Chen. *The Entity-Relationship Model: Toward a Unified View of Data*. ACM Transactions on Database Systems 1(1):9-36, 1976.

[12] K. Kulkarni and J.-E. Michels. *Temporal Features in SQL:2011*. ACM SIGMOD Record 41(3):34-43, 2012.

[13] K. Weinberger, A. Dasgupta, J. Langford, A. Smola, and J. Attenberg. *Feature Hashing for Large Scale Multitask Learning*. ICML 2009.

[14] C. Sciavolino, Z. Zhong, J. Lee, and D. Chen. *Simple Entity-Centric Questions Challenge Dense Retrievers*. EMNLP 2021. arXiv:2109.08535.